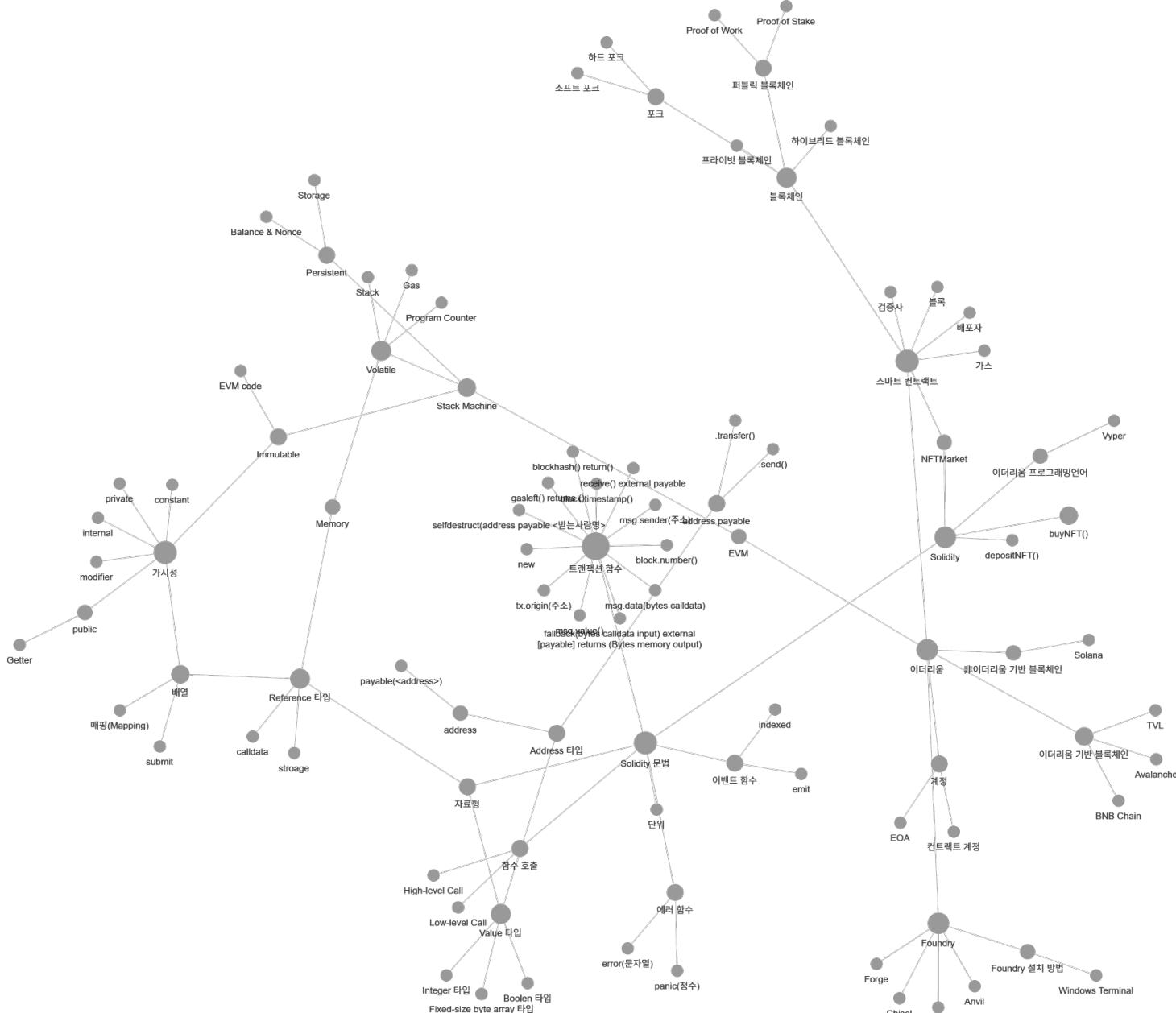




트랜잭션 함수

- `[[blockhash() return()]]`
 - `blockhash(uint 블록번지)` returns (블록주소)
 - 블록의 해쉬를 반환
 - 현재 블록 제외한 최근 256 블록에서 반환
- `[[block.number()]]`
 - 현재 블록의 넘버를 반환
- `[[block.timestamp()]]`
 - 현재 블록의 타임스탬프(로그 시간) 반환
- `[[gasleft() returns ()]]`
 - 현 함수 호출 시점 남은 가스량 반환
- `[[msg.sender(주소)]]`
 - 현재 이 메시지를 보낸 계정 정보 반환
- `[[msg.value()]]`
 - 이 메시지에 동봉된 이더의 양을 `wei` 단위로 반환
- `[[msg.data(bytes calldata)]]`
 - 메시지 데이터 반환
- `[[tx.origin(주소)]]`
 - 이 트랜잭션을 보낸 계정 정보 반환
- `[[selfdestruct(address payable <받는사람명>)]]`
 - 컨트랙트가 소지한 모든 이더가 <받는사람명>에게 넘어간 후 해당 컨트랙트 파괴
- `[[receive() external payable]]`
 - `msg.data` 없이 이더만 송금받는 경우 실행
 - external 공개 범위에 형변환(`payable`) 실행 필요
- `[[fallback(bytes calldata input) external [payable] returns (Bytes memory output)]]`
 - `msg.data` 분석 시 매칭되는 함수 없는 경우 이 함수에서 핸들링

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract Contract {
5     uint immutable a;
6     address lastSender;
7     bytes lastData;
8
9     constructor (uint value) {
10         a = value;
11     }
12
13     receive () external payable {
14         lastSender = msg.sender;
15     }
16
17     fallback (bytes calldata input) external returns (bytes memory output) {
18         lastData = input;
19         return input;
20     }
21 }
```

solidity

- `[[new]]`
 - ① 배열 동적 할당시 사용
 - ② 새로운 컨트랙트 생성시 사용

```
1 new Contract([파라미터명])
```

solidity

1 Linked Reference



Solidity 문법

- `[[트랜잭션 함수]]`

▶ Unlinked References

에러 함수

- [[panic(정수)]]
 - panic(0x00)
 - 컴파일 에러
 - panic(0x01)
 - assert 에러 (조건 에러)
 - assert(조건)
 - 조건이 거짓이라면 panic(0x01) 에러 발생
 - require(참거짓 조건, string 에러메시지)
 - 조건이 거짓이라면 Error(에러메시지) 발생
 - panic(0x02)
 - 0을 나누거나, 나머지 구할 때 발생 에러

[[error(문자열)]]

solidity

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 error InsufficientBalance(uint256 available, uint256 required);
5 #에러명 "InsufficientBalance 선언"
6
7 contract TestToken {
8     mapping(address => uint) balance;
9     # balance라는 키-값 쌍 선언
10    function transfer(address to, uint256 amount) public {
11        if (amount > balance[msg.sender])
12            # 송신액이 송신자의 잔액보다 크다면!
13            revert InsufficientBalance({
14                # 'InsufficientBalance' 함수 호출하여 거절!
15                available: balance[msg.sender],
16                # 송신자의 잔액을 호출!
17                required: amount
18                # 필요한 잔액을 'amount' 호출하여 표현!
19            });
20        balance[msg.sender] -= amount;
21        balance[to] += amount;
22        # 잔액에 필요한 잔액을 추가
23    }
24 }
```

에러 핸들링

solidity

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 interface Callee { function call() external returns (uint); }
5
6 contract FeedConsumer {
7     Callee callee;
8     uint errorCount;
9     function callCallee() public returns (bool, uint){
10         try callee.call() returns (uint v) {
11             return (true, v);
12             # bool이 성공이라면 true, v를 반환
13             # 아니라면, 아래 구문 실행
14         } catch Error(string memory reason) {
15             errorCount++;
16             return (false, 0);
17             # 조건에 따라 실패코드 0 반환
18         } catch Panic(uint /*code*/) {
19             errorCount++;
20             return (false, 1);
21             # 조건에 따라 실패코드 1반환
22         } catch (bytes memory /*LowLevelData*/) {
23             errorCount++;
24             # 에러 카운트 1 증가
25             return (false, 2);
26             # 조건에 따라 실패코드 2반환
27         }
28     }
29 }
```

Solidity 문법

- [[에러 함수]]

스마트 컨트랙트

- 블록체인 기반 자동화된 계약
- [\[\[가스\]\]](#)
 - 이더리움 화폐 이더(Ether) 토큰 가스 수수료를 검증자에게 지불
- [\[\[검증자\]\]](#)
 - 네트워크 참여들이 블록 생성하고 검증 뒤 네트워크에 배포
- [\[\[블록\]\]](#)
 - 거래(트랜잭션) 내용이 영구적으로 저장
 - 이전 블록 해시 포함하여 위/변조 어려움
- [\[\[배포자\]\]](#)
 - 스마트 컨트랙트를 네트워크상에 배포
 - 거래 통해 수수료 취함
 - 제 3자, 판매자, 구매자 모두 배포자가 될 수 있음
- [\[\[NFTMarket\]\]](#)
 - NTF를 사고 팔 수 있는 거래소 구현한 스마트 컨트랙트
- [\[\[이더리움\]\]](#)

1 Linked Reference



블록체인

- [\[\[스마트 컨트랙트\]\]](#)

▶ Unlinked References

Solidity

- C++ 기반 언어
 - 컴파일 후 이더리움 내부 바이트코드로 변환
 - Ethereum Virtual Machine(EVM)에서 해석
- **constructor()**
 - 스마트 컨트랙트 생성자
 - NFT 계약의 주소와 종개인 지정
- [\[\[depositNFT\(\)\]\]](#)
 - NFT 판매 함수
- [\[\[buyNFT\(\)\]\]](#)
 - NFT 구매 함수
- [\[\[Solidity 문법\]\]](#)

3 Linked References



NFTMarket

- [\[\[Solidity\]\]](#)
 - 스마트 컨트랙트 개발 위한 언어
 - NFTMarket 이란 스마트 컨트랙트 개발 언어

buyNFT()

- **require(msg.value == (NFT_price + fee) #Solidity #buyNFT()**
 - buyNFT()로 NFT 구매시 판매 가격 + 수수료 포함된 가격(1+0.1)을 보냄

이더리움 프로그래밍언어

- [\[\[Solidity\]\]](#)

▶ Unlinked References

가시성

- `[[public]]`
- `[[internal]]`
- `[[private]]`
- `[[constant]]`
 - 컨트랙트 외부에도 정의 가능
- `[[Immutable]]`

- solidity

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 uint constant outsideValue = 12341234;
5
6 contract Constants {
7     string constant text = "Dreamhack"; #컴파일시 값 변동
8     uint immutable x; # 컨트랙트 생성시 값 확정
9     uint constant y = 123;
10
11     constructor () {
12         x = 0x456;
13     }
14 }
```

- `[[modifier]]`

solidity

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract SimpleOwnableWithModifier {
5     address constant public owner = address(0x1234);
6     # constant : 선언 시 값 변경 불가
7     # public : 외부 컨트랙트 & 트랜잭션에서 접근 가능
8     # owner : 선언된 변수명
9     # =address(0x1234) : 메모리 0x1234 위치를 owner에 할당
10
11     modifier onlyOwner() {
12         require(msg.sender == owner, "No hack");
13         _;
14     }
15     # only : 변수명 owner에만 적용
16     # require(msg.sender==owner, "No hack") : sender가 owner가 아니면 "No hack" 반환
17     # _; : 본문의 함수를 여기(언더바)에 적용
18
19     function destroy() external onlyOwner {
20         selfdestruct(payable(msg.sender));
21     }
22 }
23 # 외부에서만 owner만 접근 가능한 onlyOwner를 호출하도록 destroy함수 선언
24 # sender에게 잔여 이더를 반환(송금) 후, 컨트랙트 자멸하도록 설계.
25 # payable 형태로 형변환해서 전송.
```

1 Linked Reference



배열

- `[[가시성]]` (접근할 수 있는 범위)

접근 범위	의미
<code>public</code>	①다른 컨트랙트&트랜잭션 등 외부에서 접근 가능
<code>internal</code>	①외부 접근 불가. ②내부&파생 컨트랙트에서 접근 가능
<code>private</code>	①외부 접근 불가.② 자신을 상속하는 파생 컨트랙트에서 접근 불가③컨트랙트 내부에서 접근 가능
<code>constant</code>	변수 값이 컴파일할 때 결정
<code>immutable</code>	변수 값은 컨트랙트 생성시 결정
<code>external</code>	①다른 컨트랙트&트랜잭션 등 외부에서 접근 가능. 단, 같은 컨트랙트 내에서 내부 호출이 불가 <code>this.<함수명>()</code> 로 호출 가능
<code>view</code>	컨트랙트의 상태 변수를 참조 하되 수정 하지 않을때 사용
<code>pure</code>	컨트랙트의 상태 변수를 수정하거나 참조하지 않을때 사용
<code>modifier</code>	함수 동작 수정. 상속&오버라이딩 가능

자료형

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract Solve {
5     function test_value() public pure returns (uint8) {
6         uint8 a = 1;
7         uint8 b = a;
8         b = 2;
9         return a;
10    }
11    function test_ref() public pure returns (uint8) {
12        uint8[3] memory a = [1, 2, 3];
13        uint8[3] memory b = a;
14        b[1] = 7;
15        return a[1];
16    }
17 }
```

• [[Value 타입]]

- 다른 변수에 할당하면 값 복사되는 타입
- 위 코드의 `test_value()` 에 해당
 - b에 a 값 복사 했을 뿐, 두 변수가 같은 값을 가르키지 않음

• [[Reference 타입]]

- 다른 변수에 할당하면 값 복사되지 않고 변수 혹은 인자가 같은 값을 가르킴
- 위 코드의 `test_ref()` 에 해당
 - b=a 이후, a와 b 두 변수가 같은 배열 가르켜서, a=[1,2,3] 이 b[1]=7 이후 a = [7,2,3]으로 변경

1 Linked Reference

Solidity 문법

- [[자료형]]

Reference 타입

- Solidity는 타 언어와 달리 Reference 타입에 변수 저장 위치 지정 필요
- Reference 타입 저장 위치 종류
 - `[[Memory]]`
 - 항상 데이터를 EVM 메모리 저장공간에 저장
 - 아래 코드에서 `uint[] memory x = data` 가 실행되어도 `data[0]` 에 미영향

- `[[calldata]]`

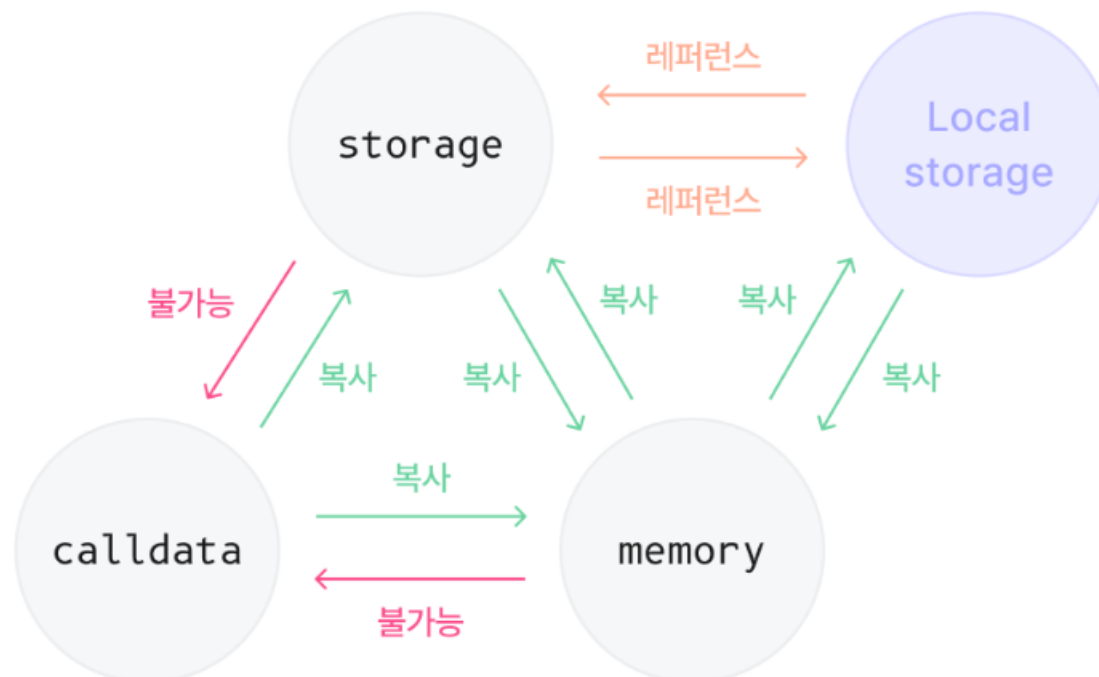
- 함수 호출시에만 발생, 임의로 생성/수정 불가

- `[[storage]]`

- 로컬 스토리지/현재 실행 중 컨트랙트의 스토리지에 담긴 데이터 가르킴
 - 아래 코드에서 `uint[] storage x = data` 가 실행되면 `data[0]` 값 수정됨

- SPDX-License-Identifier: MIT

```
pragma solidity ^0.8.0;
contract Contract {
    uint[] data = [0x1337];
    function useMemory() public returns (uint) {
        uint[] memory x = data;
        x[0] += 0x1337;
        return x[0];
    }
    function useStorage() public returns (uint) {
        uint[] storage x = data;
        x[0] += 0x1337;
        return x[0];
    }
    function checkFirstElement() public view returns (uint) {
        return data[0];
    }
}
```



- `[[배열]]`

```
uint[] data = [0x1337];
uint[] memory x = data;
x[0] += 0x1337;
return x[0];
```

Value 타입

- [[Boolean 타입]]
 - `bool`
- [[Integer 타입]]
 - `int`
 - `int16`
 - `uint` (Unsigned Integers; 부호 없는 정수형)
 - `unit8`
- [[Fixed-size byte array 타입]]
 - 가변된 bytes에 해당
 - `bytes1`
 - `bytes32`
- [[Address 타입]]
 - 이더리움 계정 주소 나타내기 위해 자료형 20바이트 크기
-

1 Linked Reference



자료형

- [[Value 타입]]
 - 다른 변수에 할당하면 값 복사되는 타입
 - 위 코드의 `test_value()` 에 해당
 - b에 a 값 복사 했을 뿐, 두 변수가 같은 값을 가르키지 않음

▶ Unlinked References